Mathematical Algorithms part 1

Day 80
28/09/2025

Why Mathematical Algorithms Are Important ?

1. Foundation of problem Solving

- Algorithms are step By step methods to Solve problems
- Mathematics gives structure, logic and proof that this steps actually Work
- Example : Euclid's Algorithm for HCD (finding greatest common divisor) is 2000+ years old and still used In Cryptography today

2. Efficiency and optimization

- In real life, resource are limited (time, memory, money)
- Mathematical algorithms help us to find the most Efficient way to Solve problems
- Example : Dijkstra's Algorithm finds the shortest path in Maps

3. Accuracy and readability

- Math Ensures that Algorithms give Correct and predictable Result's
- Without Mathematical grounding, an algorithm might fail in Edge cases.
- Example : Sorting Algorithms rely on mathematical comparisons to Ensure data is arranged perfectly Every time

4. Scalability

- Algorithms with Mathematical Analysis let us predict performance When data grows
- Example : Binary Search ($O(\log n)$) is much faster then linear Search ($O(n)$) on big data

- This is why Search Engines, databases, and apps run fast Even with billions of records.

5. Real World Applications

- Computer science → AI, Machine Learning, Data Security
- Finance → Risk Analysis, fraud detection, stock prediction
- Healthcare → Medical Image processing, drug discovery
- Engineering → Signal processing, control system, robotics.

6. Innovation

6. Innovation

- Almost every major tech breakthrough is powered by mathematical algorithms.
- Example: **RSA Algorithm** (number theory + prime factorization) made secure online transactions possible.

✅ **In short:**

Mathematical algorithms are important because they make solutions **logical, efficient, scalable, and reliable** — they're the invisible engines behind technology, science, and daily life.

How to find the GCD or HCF of Two Numbers ?

The GCD (Greatest common divisor) or HCF (Highest Common Factors) of Two Number is the largest Number that divides Both of them.

$$36 = 2 \times 2 \times 3 \times 3$$
$$60 = 2 \times 2 \times 3 \times 5$$

GCD = Multiplication of Common Factors

$$= 2 \times 2 \times 3$$
$$= 12$$

Approach 1 using loop

Step 1 Select min Number among (a, b)
result = min (a, b)

Step 2 untill result is greater then 0

Check weather (a % result == 0 && b % result == 0)
if true break.
else decrement result --

Step 3 return result

```
static int gcd(int a, int b)
{
    // Find Minimum of a and b
    int result = Math.min(a, b);
    while (result > 0) {
        if (a % result == 0 && b % result == 0) {
            break;
        }
        result--;
    }

    // Return gcd of a and b
    return result;
}
```

```
    public static void main(String[] args)
    {
        int a = 20, b = 28;
        System.out.print(gcd(a, b));
    }
}
```

output 4

Approach :

Euclidian Algorithm using Substraction O(min(a,b)) Time and O(min(a,b)) space complexity

The idea of this algorithm is, the GCD of two Numbers doesn't change if the Smaller Number is Substracted from Bigger number. This is the Euclidean algorithm By substraction. It is a process of repeat substraction, carrying the result forward Each time until the result is Equal to any one number being substracted

Psuedo code

```
gcd(a, b)
  if (a == 0) return b
  if (b == 0) return a
  if (a == b) return a          ] Base Case

  if (a > b) return gcd(a-b, b);
  else return gcd(a, b-a);
```

```
static int gcd(int a, int b) {
    // Everything divides 0
    if (a == 0)
        return b;
    if (b == 0)
        return a;

    // Base case
    if (a == b)
        return a;

    // a is greater
    if (a > b)
        return gcd(a - b, b);
    return gcd(a, b - a);
}
```

```
public static void main(String[] args) {
    int a = 20, b = 28;
    System.out.println(gcd(a, b));
}
```

output 4

The Above method can be optimised Based on following idea.

If we Notice previous approach, we can See at Some point one Number Becomes a factor of the other Number So instead of repeatedly Substracting till Both Becomes Equal, we can can check if it is a factor using modulus Operator

Consider a = 98 and b = 56

a = 98, b = 56:

- *a > b so put a = a-b and b remains same. So a = 98-56 = 42 & b= 56.*

a = 42, b = 56:

- *Since b > a, we check if b%a=0. Since answer is no, we proceed further.*
- *Now b>a. So b = b-a and a remains same. So b = 56-42 = 14 & a= 42.*

a = 42, b = 14:

- *Since a>b, we check if a%b=0. Now the answer is yes.*
- *So we print smaller among a and b as H.C.F . i.e. 42 is 3 times of 14.*

```
static int gcd(int a, int b) {
    // Everything divides 0
    if (a == 0)
        return b;          // division By Zero check
    if (b == 0)
        return a;

    // Base case
    if (a == b)
        return a;

    // a is greater
    if (a > b) {
        if (a % b == 0)    // check for factor
            return b;
        return gcd(a - b, b);
    }

    // b is greater
    if (b % a == 0)        // check for factor
        return a;
    return gcd(a, b - a);
}
```

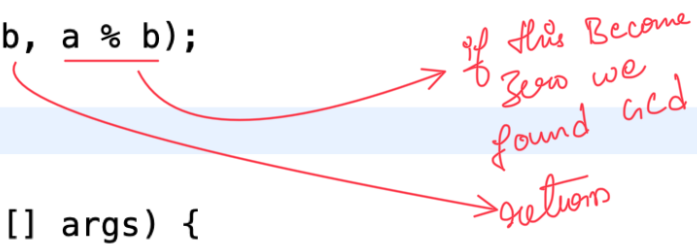
Better Approach

The algorithm is based on the below facts.

- *If we subtract a smaller number from a larger one (we reduce a larger number), GCD doesn't change. So if we keep subtracting repeatedly the larger of two, we end up with GCD.*
- *Now instead of subtraction, if we divide the larger number, the algorithm stops when we find the remainder 0.*

```java
// Recursive function to calculate GCD using Euclidean algorithm
static int gcd(int a, int b) {
    return (b == 0) ? a : gcd(b, a % b);
}

// Driver code
public static void main(String[] args) {
    int a = 20, b = 28;
    System.out.println(gcd(a, b));
}
```



Time Complexity: O(log(min(a,b)))

- Each recursive call reduces the size of the numbers significantly using the modulo operation (`a % b`), which shrinks the input faster than subtraction.
- The worst-case scenario for the number of steps occurs when the inputs are consecutive Fibonacci numbers, like (21, 13), which maximizes the number of recursive calls.
- Since Fibonacci numbers grow exponentially, and the number of steps increases linearly with their position, the time complexity becomes logarithmic in terms of the smaller number — O(log(min(a, b))).

Auxiliary Space: O(log(min(a,b))

- The maximum number of recursive calls is proportional to the number of steps taken to reduce the input to zero, which is O(log(min(a, b))) in the worst case.

Coming days I will Explore more Mathamatical Algorithms